PARSER

GTECH 3 - 2025/2026

OPENGL

SHADER++

```
#version 330 core
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec2 aTexCoor;
layout (location = 2) in vec3 aNormal;

out VERTEX_OUT {
    vec3 FragPos;
    vec3 Normal;
    vec2 TexCoords;
    vec3 LightPos;
    vec3 ViewPos;
} v;

uniform mat4 _projection;
uniform mat4 _view;
uniform mat4 _model;
uniform vec3 _lightPos;

void main()
{
    gl_Position = _projection * _view * _model * vec4(aPos, 1.0);
    v.TexCoords = vec2(aTexCoor.x, 1-aTexCoor.y);
    v.FragPos = vec3(_view * _model * vec4(aPos, 1.0));
    v.Normal = vec3(_view * transpose(inverse(_model)) * vec4(aNormal, 1.0));
    v.LightPos = vec3(inverse(_view) * vec4(_lightPos, 1));
    v.ViewPos = vec3(_view * vec4(0, 0, 0, 1));
}
```

```
#version 330 core
out vec4 FragColor;

in VERTEX_OUT {
    vec3 FragPos;
    vec3 Normal;
    vec2 TexCoords;
    vec3 LightPos;
    vec3 ViewPos;
} v;

uniform sampler2D _texture;
uniform vec4 _mainColor;
uniform vec4 _lightColor;
uniform float _ambientStrength;

void main()
{
    vec4 ambient = _ambientStrength * _lightColor;

    vec3 norm = normalize(v.Normal);
    vec3 lightDir = normalize(v.LightPos - v.FragPos);
    float diff = max(dot(norm, lightDir), 0.0);
    vec4 diffuse = diff * _lightColor;

    float specularStrength = 0.5;
    vec3 viewDir = normalize(-v.FragPos);
    vec3 reflectDir = reflect(-lightDir, norm);
    float spec = pow(max(dot(viewDir, reflectDir), 0.0), 32);
    vec4 specular = specularStrength * spec * _lightColor;

    vec4 lightFactor = (ambient + diffuse + specular);

    ivec2 decalSize = textureSize(_texture, 0);
    if (decalSize != ivec2(1, 1))
        FragColor = _mainColor * texture(_texture, v.TexCoords) * lightFactor;
    else
        FragColor = _mainColor * lightFactor;
}
```

UTILISATION

```
mngl::Shader shader( vertexPath: "shader/vertex.glsl", fragmentPath: "shader/fragment.glsl");
shader.SetMatrix4( name: "_projection", value: glm::perspective( fovy: glm::radians( degrees: 75.f), aspect: 800.f/600.f, zNear: 0.1f, zFar: 100.f));
shader.SetMatrix4( name: "_view", value: glm::translate( m: glm::mat4( s: 1.f), v: {x: 0, y: 0, z: -3.f}));
shader.SetMatrix4( name: "_model", value: glm::mat4( s: 1.f));
shader.SetColor( name: "_mainColor", value: mngl::Color::Red);
shader.SetColor( name: "_lightColor", value: mngl::Color::White);
shader.SetFloat( name: "_ambientStrength", value: 0.1f);
shader.SetVector3( name: "_lightPos", value: {x: 0, y: 0, z: -3.f});
```

```
void mngl::Shader::SetColor(const std::string& _name, const Color& _value) const
{
    Use();
    glUniform4f(glGetUniformLocation(m_id, name: _name.c_str()), v0: _value.r, v1: _value.g, v2: _value.b, v3: _value.a);
    Shutdown();
}

void mngl::Shader::SetVector3(const std::string& _name, const glm::vec3& _value) const
{
    Use();
    glUniform3f(glGetUniformLocation(m_id, name: _name.c_str()), v0: _value.x, v1: _value.y, v2: _value.z);
    Shutdown();
}

void mngl::Shader::SetMatrix4(const std::string& _name, const glm::mat4& _value) const
{
    Use();
    glUniformMatrix4fv(glGetUniformLocation(m_id, name: _name.c_str()), count: 1, transpose: GL_FALSE, glm::value_ptr(_value));
    Shutdown();
}
```

NORMAL

```
glm::vec3 p1 = {x: -0.5f, y: -0.5f, z: 0.0f};
glm::vec3 p2 = {x: 0.5f, y: -0.5f, z: 0.0f};
glm::vec3 p3 = {x: 0.0f, y: 0.5f, z: 0.0f};

glm::vec3 v = p2 - p1;
glm::vec3 u = p3 - p1;

glm::vec3 normal = glm::cross(x: v, y: u);

float allVertice[] = {
    p1.x, p1.y, p1.z,
    normal.x, normal.y, normal.z,
    p2.x, p2.y, p2.z,
    normal.x, normal.y, normal.z,
    p3.x, p3.y, p3.z,
    normal.x, normal.y, normal.z
};

unsigned int VBO, VAO;

glGenVertexArrays(n: 1, &VAO);
glGenBuffers(n: 1, &VBO);

glBindVertexArray(VAO);

glBindBuffer(target: GL_ARRAY_BUFFER, buffer: VBO);
glBufferData(target: GL_ARRAY_BUFFER, size: sizeof(allVertice), allVertice, usage: GL_STATIC_DRAW);

glVertexAttribPointer(index: 0, size: 3, type: GL_FLOAT, normalized: GL_FALSE, stride: 6 * sizeof(float), (void*)0);
glEnableVertexAttribArray(index: 0);

glVertexAttribPointer(index: 2, size: 3, type: GL_FLOAT, normalized: GL_FALSE, stride: 6 * sizeof(float), (void*)(3*sizeof(float)));
glEnableVertexAttribArray(index: 2);

glBindBuffer(target: GL_ARRAY_BUFFER, buffer: 0);
glBindVertexArray(array: 0);
```

EXERCICE

ON AFFICHE LE .OBJ

- Récupérez le projet basicgl envoyer sur discord.
- Rajoutez votre loader de .obj.
- Afficher le .obj.

.FBX

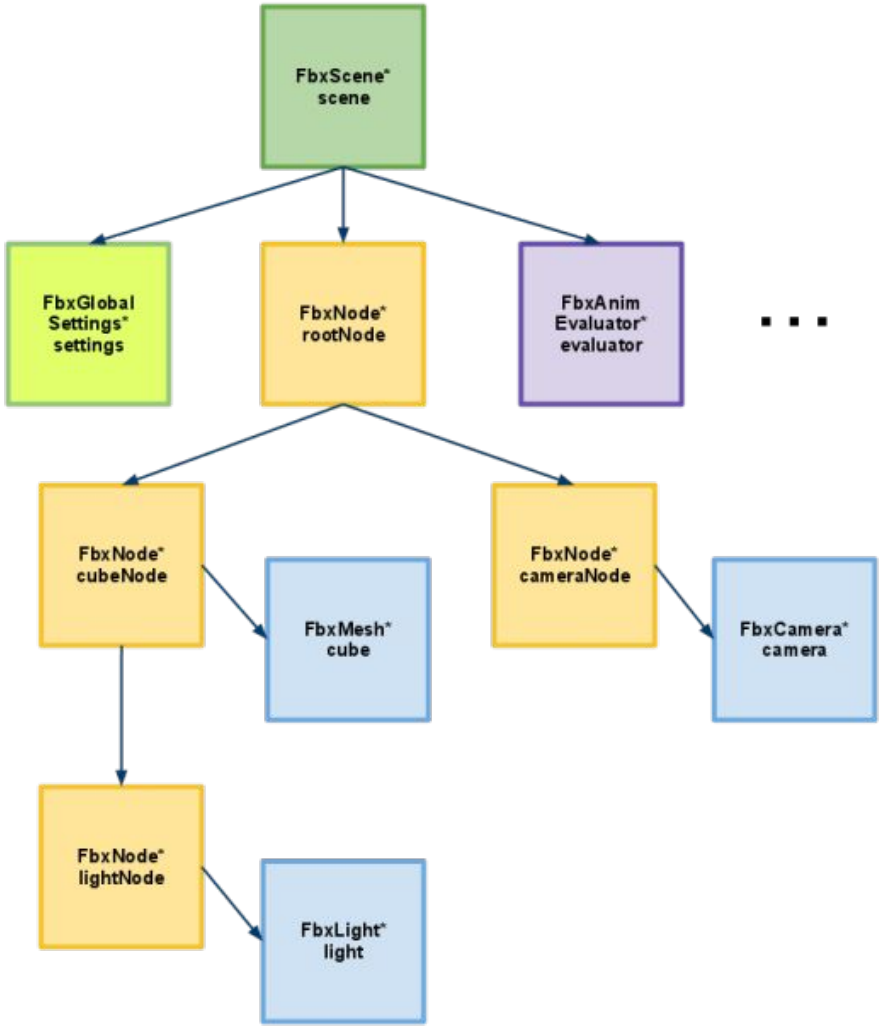
STRUCTURE

The FBX SDK scene graph is abstracted by the FbxScene class.

The scene is organized as a hierarchy of nodes (FbxNode).

The root node of the scene is accessed via FbxScene::GetRootNode().

A scene element, for example, a mesh, a light, or a camera is defined by combining a FbxNode with a subclass of FbxNodeAttribute.



INSTALLATION

Documentation: https://help.autodesk.com/view/FBX/2020/ENU/

Installez le sdk en suivant ce lien:
https://help.autodesk.com/view/FBX/2020/ENU/?guid=FBX_Developer_Help_getting_started_installing_and_configuring_configuring_the_fbx_sdk_for_wind_html

Installation avec cmake:
- Copier le dossier 2020.3.7 à la racine.
- Renomez le fbxsdk.
- Recopier les propriété partager sur discord.

DEMO SDK

EXERCICE

TOUT VA MAL ?

- Ajouter le sdk fbx dans le projet cmake basicgl
- Faire un bout de code pour loader les informations du .fbx
- Créer le tableau qui agglomère toute les informations de vertex par face
- Afficher le résultat

ATTENTION: les fbxNode ont un transform.
Le site de kenney.nl a des fbx simple.